MSML606 Final Exam Information

Date and Time: Thursday, 5/14, 4:00pm-6:00 pm

Duration: 2 hours

Format: Pen and paper

Exam types:

- True/False
- Multiple choice (select the single best answer)
- Multiple answers (select all that apply, penalty applies for incorrect selections)
- Short answers
- Certain questions will explicitly require you to demonstrate your process on your scratch paper. For these items, clear and readable derivations must be shown to receive credit.

What's allowed:

- Two (2) study sheets: Standard letter size (8.5x11 in). You may use both sides for text, diagrams, or formulas. Handwritten or typed.
- One (1) scratch paper: Standard letter size (8.5x11 in). Use this to show your work.
- To receive credit for the exam, you must submit both your study sheet and your scratch paper (if used) along with your exam. Write your name clearly on both.

Important Policies & Conduct

- Seating: You must leave at least one empty seat between you and your neighbor.
- Electronic Devices: All phones, smartwatches, and digital devices must be completely powered off (not silent/vibrate) and placed face-down on the far corner of your desk. They must remain visible and untouched.
- Collaboration: Any form of communication, texting, or use of AI/online tools is strictly prohibited.
- No Talking: The exam room must remain silent. If a conversation occurs, both parties will receive a score of 0, regardless of the topic.
- Bathroom Breaks: No bathroom breaks are permitted during the examination period. Please plan accordingly before entering the room.

Important: Any violation of these restrictions or any act of cheating (whether attempted or succeeded) will result in a report to the University’s Office of Student Conduct.

Study guide

Main resources (highest priority):

Thoroughly review the lecture slides, quizzes, and assignments (including the ungraded ones). These are your primary study materials. The exam is designed around what we covered this semester, and many questions focus on understanding, walking through algorithms, and applying concepts to real scenarios, **not** memorizing formulas.

Secondary resources (helpful but not representative):

Last semester's exams, and additional resources listed in the slides can help you reinforce your understanding.

Important note: These are not sample exams for this semester. They are meant to give you extra practice, not to predict the exact style or content of this exam.

To succeed, you must move beyond simple memorization and be prepared to demonstrate the following:

- Foundational knowledge: Be familiar with all terminology, definitions, and mathematical properties
- Performance analysis: Understand time complexity, trade-offs, and comparisons with alternative methods
- Algorithmic logic: Explain the "Why" and "How" of an algorithm, other than just identifying its name
- Concrete application: Perform manual computations, tracing, or walkthroughs on small, specific examples
- Critical Comparisons: Identify pros, cons, and trade-offs between similar techniques (e.g., Hashing vs. BSTs).

For all equations and formulas covered in lecture:

- Know what each variable and parameters represents and how changing them affects the result.
- Input vs. Output: Clearly distinguish what the formula takes in and what it produces.
- Match a technical description or real-world scenario to its mathematical representation.

Coding & Pseudocode Expectations

- You are not required to write long pseudocode or complex functions from scratch.
- However, you are expected to understand the algorithms used in the homework at the source-code level and be able to demonstrate how they work
- Exam questions may test your understanding of algorithmic processes. Examples include (but not limited to):
    - Fill in missing logic or blanks in a provided algorithm
    - Track and record the intermediate values of variables during execution
    - Write simple operations (e.g., single "if" conditions for a BST search)
    - Debugging: Identify a "bug" or a missing edge cases in a provided algorithm (only if covered in lecture slides)
    - Describe the step-by-step transformation of data from input to output for any given method

Final Exam Topics

(This is not an exhaustive list; it reflects the major themes covered in lecture)

Lecture 11 Quick sort

- All slides

Lecture 12 Heap and Heap sort, priority queues

- Heap properties
- Array representation of binary trees and basic operations
- All heap operations: step by step behavior & time complexities
- Heapsort: full procedure and key properties
- Group activity

Lecture 13

- Order statistic
- *Skip: Finding minimum and finding min and max*
- Selection problem
- Partition function

- Quicksort vs. Quickselect
- Randomized-select: algorithm and analysis
- Median of medians: algorithm and analysis

Lecture 14 NP Completeness

- Tractability
- NP-complete and np-hard problems examples, and why they belong to these categories
- Decision & optimization problems
- Definitions: P, NP, NP-complete, NP-hard
- Deterministic vs. Nondeterministic machines
- Reduction (high-level idea)
- P=NP
- Approaches for NP-complete/NP-hard problems
- Transforming intractable problems with machine learning and AI
- Discussion questions in the deck
- *Skip:*
    - *proving NP-hardness – restriction, local replacement, component design*
    - *Complexity classes (last slide)*

Lecture 15-16 Graphs

- All graph terms, definitions, properties
- Graph representations: adjacency matrix, adjacency list
- BFS algorithm and analysis
- BFS Tree
- Shortest path problem definition
- Proof: What must be shown to prove that BFS is correct for the shortest-path problem?
- DFS algorithm and analysis
- DFS trees/forest

Lecture 17-18 DFS Topological sort and other applications

- Parenthesis theorem

- White-path theorem (high-level idea): DFS will discover every vertex that is reachable and still unvisited from a starting vertex u at the moment u is discovered
- Classification of edges
- Edge Classification using DFS
- Cycle detection using DFS
- Topological sort problem definition
- Directed acyclic graph (DAG)
- Topological sort algorithm (no pseudocode required but should be able to demonstrate the step-by-step process)
- Interpreting the topological sort output
- Analysis of the algorithm
- Strongly connected components
    - Definition and assumptions
    - Identifying strongly components in graphs
    - How DFS is used in SCC algorithm
- Articulation points
    - Definitions
    - Identify articulation points in a graph

Lecture 19-20 Dijkstra and MST

- Dijkstra
    - Definitions for shortest paths
    - Relaxation
    - Pseudocode and step-by-step walkthrough
    - BFS vs. Dijkstra
    - Skip correctness proof
    - Heap data structure in Dijkstra: how it affects time complexity
    - Assumptions for Dijkstra algorithm
- MST
    - Definitions
    - Cut-and-paste idea
    - Kruskal's algorithm
    - Prim's algorithm

Lecture 21 Greedy Algorithms

- Optimization problems
- Components of a greedy algorithm
- Greedy algorithm examples and what makes them greedy

- Greedy-choice property
- Optimal substructure
- 0/1 knapsack
    - brute force, DFS, Backtracking, Branch and bound
- TSP
    - Greedy vs. exact solutions discussed
- All discussion questions

Lecture 22 – Dynamic programming (1)

- Overlapping subproblems
- Optimal substructure (definition and examples)
- Memorization, tabulation for Fibonacci
- Time and space complexity
- Climbing stairs, coin change – all methods and questions

Lecture 23 – Dynamic programming (2)

- Matrix multiplication
    - Problem definition
    - State, base case, and DP formula (main computation)
    - Naïve recursion algorithm
    - Naïve recursion +memorization
    - Tabulation
    - Step-by-step algorithm execution
    - Interpretating the result (how to read output tables and where to find the final answer)

- Longest Common Subsequence
    - Problem definition
    - Subsequence vs. Substring
    - Optimal substructure and subproblems
    - State and base cases
    - Recursive formula for computing the current state
    - Brute force recursive solution
    - Recursion + memorization
    - Tabulation
    - Reading and interpreting the DP Table
- DP summary